

Temas Tratados en el Trabajo Práctico 4

- Representación del Conocimiento y Razonamiento Lógico.
- Estrategias de resolución de hipótesis: Encadenamiento hacia Adelante, Encadenamiento hacia Atrás y Resolución por Contradicción.
- Representación basada en circuitos.

Ejercicios Teóricos

1. ¿Qué es una inferencia?

Es el proceso mediante el cual un agente llega a nuevas conclusiones a partir de conocimiento que ya tiene. Es decir que a partir de hechos o reglas conocidas, deduce o interfiere información nueva

Por ejemplo, si un agente sabe que "La oficina esta sucia, entonces hay que limpiarla"

Y también sabe que "La oficina esta sucia"

Entonces infiere que hay que limpiar la oficina

1. ¿Cómo se verifica que un modelo se infiere de la base de conocimientos?

Para verificar que una sentencia se infiere de la base de conocimiento, se revisa si es verdadera en todos los modelos donde la base de conocimientos también lo es. O sea que queremos saber si una conclusión es lógicamente consecuencia de lo que ya sabemos (la base de conocimiento).

1. Observe la siguiente base de conocimiento:

$R1 : b \wedge c \rightarrow a$

$R2 : d \wedge e \rightarrow b$

$R3 : g \wedge e \rightarrow b$

$R4 : e \rightarrow c$

$R5 : d$

$R6 : e$

$R7 : a \wedge g \rightarrow f$

3.1 ¿Cómo se puede probar que $a = \text{True}$ a través del encadenamiento hacia adelante? Este m

3.2 ¿Cómo se puede probar que $a = \text{True}$ a través del encadenamiento hacia atrás? Este m

3.3 Expresar la base de conocimiento en su Forma Normal Conjuntiva. A continuación, demu

3.1 -> Este método parte de los hechos conocidos ($R5$ y $R6$) y se van aplicando reglas para inferir nuevas verdades hasta llegar a la meta. Entonces por

R5 y R6 sabemos que d y e son verdaderos. Por R2, sabemos que b sera verdadero. Por R4, sabemos que c sera verdadero. Por R1, llegamos entonces a que a es verdadero, cumpliendo asi la meta. La propiedad requerida para que el algoritmo funcione correctamente es que "debe ser completo"

3.2 -> En este ahora partimos de la meta, osea que a es verdadero y retrocedemos a ver que hechos necesitamos para que esto sea asi. Por R1 para que a sea verdadero, necesitamos que b y c sea verdadero, por R2 para que b sea verdadero, d y e tienen que ser verdadero, por R4 si e es verdadero, entonces c sera verdadero. Ahora teniendo b y c verdadero por R1 se hace verdadero a, asi que se cumple. La propiedad que se necesita para que este metodo funcione bien es que "el algoritmo debe ser solido"

3.3 -> Sabiendo que la base de conocimiento es: R1: $b \wedge c \rightarrow a$ R2: $d \wedge e \rightarrow b$ R3: $g \wedge e \rightarrow b$ R4: $e \rightarrow c$ R5: d R6: e R7: $a \rightarrow g$

1. Pasar todo a la forma normal conjuntiva (FNC). La FNC es una conjunción de cláusulas (cada una disyunción de literales). La BC en FNC es: R1. $\neg b \vee \neg c \vee a$ (es la equivalencia de $b \wedge c \rightarrow a$) R2. $\neg d \vee \neg e \vee b$ (equivalencia de $d \wedge e \rightarrow b$) R3. $\neg g \vee \neg e \vee b$ (equivalencia de $g \wedge e \rightarrow b$) R4. $\neg e \vee c$ (equivalencia de $e \rightarrow c$) R5. d (ya es un literal positivo) R6. e (ya es un literal positivo) R7. $\neg a \vee \neg g$ (equivalencia de $a \rightarrow g$)
2. Demostración por contradicción de que $a = \text{True}$ Para probar que a es verdadero: Asumimos lo contrario: $\neg a$. Si la BC $\neg a$ lleva a contradicción, entonces a es verdadero.

Paso 1: Partimos de los hechos (Usamos los nuevos R no los anteriores) De R5 y R6 tenemos: d verdadero; e verdadero Paso 2: Usamos R2 sustituyendo d y e verdaderos: $(F = \text{falso}) \neg d \vee \neg e \vee b = F \vee F \vee b = b$. Entonces $\rightarrow b$ es verdadero Paso 3: Usamos R4 Con e verdadero: $\neg e \vee c = F \vee c = c$. Entonces $\rightarrow c$ es verdadero Paso 4: Usamos R1 Con b y c verdaderos: $\neg b \vee \neg c \vee a = F \vee F \vee a = a$. Entonces $\rightarrow a$ es verdadero

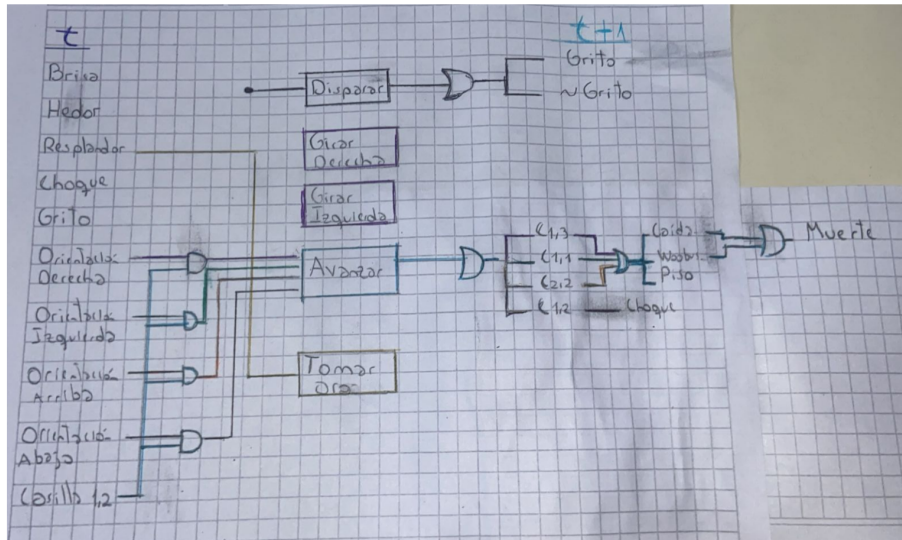
1. Contradicción Primero asumimos que " $\neg a$ " era verdadero como suposición pero llegamos a que "a" es verdadero por derivación lógica generando una contradicción con la suposición " $\neg a$ ". Por lo tanto por reducción al absurdo, la BC $\neg a$ es inconsistente. Por lo tanto: $a = \text{True}$
1. Diseñe con lógica proposicional basada en circuitos las proposiciones *Orientado Derecha* y *Agente ubicado en la casilla [1,2]* para el mundo de wumpus de 4x4. Dibuje el circuito correspondiente.

```
from PIL import Image
import matplotlib.pyplot as plt

# Abrir la imagen desde la carpeta del proyecto
img = Image.open("tp4_4.png")

# Mostrar la imagen con matplotlib
```

```
plt.figure(figsize=(img.width/80, img.height/80)) # Ajustá el divisor para el tamaño
plt.imshow(img)
plt.axis("off")
plt.show()
```



1. El nonograma es un juego en el cual se posee un tablero en blanco y cada fila y columna presenta información sobre la longitud de un bloque en dicha fila/columna. Además, la leyenda puede indicar más de un número, indicando esto que existen varios bloques de las longitudes mostradas por la leyenda y en el mismo orden, separados por al menos un espacio vacío.

Resuelva el nonograma de la imagen de abajo escribiendo en primer lugar cada regla que puede incorporarse a la base de conocimientos inicial e incorporando cada inferencia que realice.

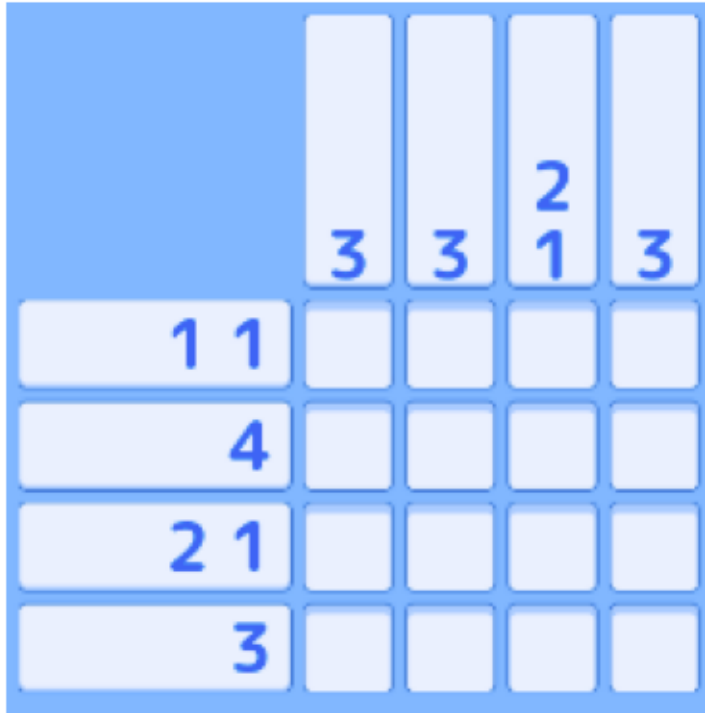
```
import requests
from PIL import Image
from io import BytesIO
import matplotlib.pyplot as plt

# URL directa de Google Drive
url = "https://drive.google.com/uc?export=view&id=1SKiXvrI_TX-U4sbw60TYSRmaNYyFixmI"

# Descargar la imagen
response = requests.get(url)
img = Image.open(BytesIO(response.content))

# Mostrar la imagen
plt.imshow(img)
```

```
plt.axis('off') # Ocultar ejes
plt.show()
```



```
from PIL import Image
import matplotlib.pyplot as plt

# Abrir la imagen desde la carpeta del proyecto
img = Image.open("tp4_5.png")

# Mostrar la imagen con matplotlib
plt.figure(figsize=(img.width/80, img.height/80)) # Ajustá el divisor para el tamaño
plt.imshow(img)
plt.axis("off")
plt.show()
```

$$R1: (C_{11} \wedge C_{13}) \vee (\cancel{C_{12} \wedge C_{14}}) \vee (\cancel{C_{11} \wedge C_{14}})$$

- R1. 1: C_{11}
- R1. 2: $\neg C_{12}$
- R1. 3: $\neg C_{14}$

$$R2: (C_{21} \wedge C_{22} \wedge C_{23} \wedge C_{24})$$

- R2. 1: C_{21}
- R2. 2: C_{22}
- R2. 3: C_{23}
- R2. 4: C_{24}

$$R3: (C_{31} \wedge C_{32} \wedge C_{34}) \vee (\cancel{C_{31} \wedge C_{33} \wedge C_{34}})$$

- R3. 1: C_{32}
- R3. 2: C_{34}
- R3. 3: $\neg C_{33}$

$$R4: (\cancel{C_{41} \wedge C_{42} \wedge C_{43}}) \vee (C_{42} \wedge C_{43} \wedge C_{44})$$

- R4. 1: C_{42}
- R4. 2: C_{43}
- R4. 3: C_{44}

$$R5: (C_{11} \wedge C_{21} \wedge C_{31}) \vee (\cancel{C_{21} \wedge C_{31} \wedge C_{41}})$$

- R5. 1: C_{31}
- R5. 2: $\neg C_{41}$

$$R6: (\cancel{C_{12} \wedge C_{22} \wedge C_{32}}) \vee (C_{22} \wedge C_{32} \wedge C_{42})$$

- R6. 1: C_{22}
- R6. 2: C_{32}
- R6. 3: C_{42}

$$R7: (C_{13} \wedge C_{23} \wedge C_{43}) \vee (\cancel{C_{13} \wedge C_{33} \wedge C_{43}})$$

- R7. 1: C_{13}
- R7. 2: C_{43}
- R7. 3: $\neg C_{33}$

$$R8: (\cancel{C_{14} \wedge C_{24} \wedge C_{34}}) \vee (C_{24} \wedge C_{34} \wedge C_{44})$$

- R8. 1: C_{24}
- R8. 2: C_{44}

Nomograma Resuelto



Ejercicios de Implementación

1. Implementar un motor de inferencia con encadenamiento hacia adelante. Pruébalo con las proposiciones del ejercicio 3.

```
from sympy import symbols
```

```

a, b, c, d, e, f, g = symbols('a b c d e f g') #Todas las letras que van a ser usadas

BHO = {d, e} #Hechos iniciales

REGLAS = [ #Aca van las proposiciones
    ([b, c], a),
    ([d, e], b),
    ([g, e], b),
    ([e], c),
    ([a, g], f),
]

goal = a #Objetivo a verificar

def encadenamiento(facts, rules, goal=None, max_iter=10_000):

    #Aca va la logica del encadenamiento hacia adelante
    BH = set(facts)
    changed = True
    iters = 0

    while changed and iters < max_iter:
        changed = False
        iters += 1

        for antecedents, consequent in rules:
            #Si ya esta el consecuente, no hace falta disparar esta regla
            if consequent in BH:
                continue

            #Para agregan los antecedentes
            if all(a in BH for a in antecedents):
                BH.add(consequent)
                changed = True

            #Verifica si se cumple la meta, si lo hace ya corta
            if goal is not None and goal in BH:
                return BH, True

    return BH, (goal in BH if goal is not None else False)

cierre, alcanzado = encadenamiento(BHO, REGLAS, goal)

print("Hechos al finalizar:")
print(cierre)
print("Se alcanza la meta?", alcanzado)

```

Hechos al finalizar:
{a, d, b, c, e}
Se alcanza la meta? True

1. Implementar un motor de inferencia con encadenamiento hacia atrás.
Pruébelo con las proposiciones del ejercicio 3.

```
# Reglas:
#R1: b   c → a
#R2: d   e → b
#R3: g   e → b
#R4: e → c
#R5: d
#R6: e
#R7: a   g → f

print("Se busca probar que a es verdadero dado los hechos d y e mediante encadenamiento hacia atrás")

# Motor de inferencia con encadenamiento hacia atrás (comprobando todas las proposiciones)
reglas = {
    "a": [["b", "c"]],
    "b": [["d", "e"], ["g", "e"]],
    "c": [["e"]],
    "f": [["a", "g"]],
}

# Hechos iniciales
hechos = {"d", "e"}

# Función de inferencia hacia atrás
def inferir(objetivo, hechos, reglas, visitados=None):
    if visitados is None:
        visitados = set()

    # Si ya lo sabemos como hecho
    if objetivo in hechos:
        return True

    # Evitar ciclos
    if objetivo in visitados:
        return False
    visitados.add(objetivo)
```

```

    if objetivo not in reglas:
        return False

    # Probar todas las reglas posibles
    for premisas in reglas[objetivo]:
        if all(inferir(p, hechos, reglas, visitados) for p in premisas):
            hechos.add(objetivo)
            for alt in reglas.get(objetivo, []):
                hechos.update(alt)
            return True

    return False

# Lista de todas las proposiciones
proposiciones = ["a", "b", "c", "d", "e", "f", "g"]

print("Inferencia hacia atrás sobre todas las proposiciones")
print("Hechos iniciales:", hechos)

# Bucle para verificar cada proposición
resultado = {}
for prop in proposiciones:
    resultado[prop] = inferir(prop, hechos, reglas)

print("\nResultados:")
for prop in proposiciones:
    estado = "Verdadero" if resultado[prop] else "No se pudo probar"
    print(f"{prop}: {estado}")

# Verificar si todas son verdaderas
if all(resultado.values()):
    print("Se probó que a es verdadera")
    print("Todas las proposiciones se probaron como verdaderas.")
else:
    print("No se pudo probar que a es verdadera")
    print("No todas las proposiciones pudieron probarse como verdaderas.")

print("\nHechos finales deducidos:", hechos)

```

Se busca probar que a es verdadero dado los hechos d y e mediante encadenamiento hacia atrás
Inferencia hacia atrás sobre todas las proposiciones
Hechos iniciales: {'d', 'e'}

Resultados:

```

a: Verdadero
b: Verdadero
c: Verdadero
d: Verdadero
e: Verdadero
f: Verdadero
g: Verdadero
Se probó que a es verdadera
Todas las proposiciones se probaron como verdaderas.

```

Hechos finales deducidos: {'a', 'b', 'c', 'e', 'g', 'd', 'f'}

1. Implementar un motor de inferencia por contradicción que detecte si el conjunto de proposiciones del ejercicio 3 es inconsistente.

```

# - Encadenamiento hacia adelante
# - Encadenamiento hacia atrás
# - Prueba por contradicción
# - Cadena de resolución como ejemplo

```

```

from typing import List, Set, Dict
from dataclasses import dataclass
from collections import defaultdict

```

```

# ----- Base de Conocimiento (BC) -----
# Reglas Horn: (premisas, cabeza)
reglas_horn = [
    ({'b', 'c'}, 'a'), # R1: b c → a
    ({'d', 'e'}, 'b'), # R2: d e → b
    ({'g', 'e'}, 'b'), # R3: g e → b
    ({'e'}, 'c'), # R4: e → c
    (set(), 'd'), # R5: d
    (set(), 'e'), # R6: e
    ({'a', 'g'}, 'f'), # R7: a g → f
]

```

```

# Hechos

```

```

hechos_iniciales = {head for prem, head in reglas_horn if not prem}

```

```

# 1) Encadenamiento hacia adelante -----

```

```

def encadenamiento_adelante(reglas, hechos):
    hechos = set(hechos)
    pasos = []
    cambio = True
    while cambio:
        cambio = False

```

```

        for prem, cabeza in reglas:
            if prem.issubset(hechos) and cabeza not in hechos:
                hechos.add(cabeza)
                pasos.append(f"{cabeza} porque se verifica {sorted(prem)}")
                cambio = True
    return hechos, pasos

# 2) Encadenamiento hacia atrás -----
def encadenamiento_atras(meta: str, reglas, hechos):
    index = defaultdict(list)
    for prem, cabeza in reglas:
        index[cabeza].append(prem)
    for k in index:
        index[k].sort(key=len)

    memo: Dict[str, bool] = {}
    en_visita: Set[str] = set()
    pasos: List[str] = []

    def probar(g: str) -> bool:
        if g in memo:
            return memo[g]
        if g in hechos:
            pasos.append(f"{g} es un hecho")
            memo[g] = True
            return True
        if g in en_visita:
            pasos.append(f"ciclo en {g}")
            memo[g] = False
            return False

    en_visita.add(g)
    ok = False
    for prem in index.get(g, []):
        if prem:
            pasos.append(f"{g} se verifica con {sorted(prem)}")
            if all(probar(p) for p in sorted(prem)):
                pasos.append(f"{g} se verifica")
                ok = True
                break
    if not ok:
        pasos.append(f"{g} no se verifica")
    en_visita.remove(g)
    memo[g] = ok
    return ok

```

```

        return probar(meta), pasos

# 3) CNF + Resolución por contradicción -----
Literal    = str
Clausula    = frozenset
CNF          = Set[Clausula]

def neg(lit: Literal) -> Literal:
    return lit[1:] if lit.startswith("~") else "+"lit

# BC simplificada
KB_CNF: CNF = {
    frozenset({"~b", "~c", "a"}), # R1
    frozenset({"~d", "~e", "b"}), # R2
    frozenset({"~g", "~e", "b"}), # R3
    frozenset({"~e", "c"}),        # R4
    frozenset({"d"}),              # R5
    frozenset({"e"}),              # R6
    frozenset({"~a", "~g", "f"}), # R7
}

def resolver_existe_vacio(c1: Clausula, c2: Clausula) -> bool:
    """Devuelve True si alguna resolución entre c1 y c2 produce la cláusula vacía."""
    for lit in c1:
        if neg(lit) in c2:
            res = (c1 - {lit}) | (c2 - {neg(lit)})
            if len(res) == 0:
                return True
    return False

def resolucion_insat_simple(cnf: CNF, max_iter: int = 1000) -> bool:
    """
    Chequea insatisfacibilidad por resolución sin traza.
    Devuelve True si se obtiene la cláusula vacía (contradicción).
    """
    claus = set(cnf)
    vistos = set()
    for _ in range(max_iter):
        nuevos = set()
        lista = list(claus)
        hubo_nuevo = False
        for i in range(len(lista)):
            for j in range(i+1, len(lista)):
                par = (lista[i], lista[j])
                if par in vistos:

```

```

        continue
    vistos.add(par)

    if resolver_existe_vacio(lista[i], lista[j]):
        return True

    for lit in lista[i]:
        if neg(lit) in lista[j]:
            res = (lista[i] - {lit}) | (lista[j] - {neg(lit)})
            if res not in claus:
                nuevos.add(frozenset(res))
                hubo_nuevo = True

    claus |= nuevos
    if not hubo_nuevo:
        break
    return False

# Resultados
# 1)
print("1) Encadenamiento hacia adelante")
hechos_fc, pasos_fc = encadenamiento_adelante(reglas_horn, hechos_iniciales)
print("Hechos iniciales:", sorted(hechos_iniciales))
for p in pasos_fc:
    print(" -", p)
print("¿a derivado?:", "sí" if "a" in hechos_fc else "no")
print()

# 2) meta = 'a'
print("2) Encadenamiento hacia atrás (meta = 'a')")
ok, pasos_bc = encadenamiento_atras("a", reglas_horn, hechos_iniciales)
for p in pasos_bc:
    print(" -", p)
print("¿a probado?:", "sí" if ok else "no")
print()

# 3) ¿BC sola es inconsistente?
print("3) ¿BC es inconsistente?")
insat_kb = resolucion_insat_simple(KB_CNF)
print("Resultado:", "Si es INCONSISTENTE porque hay contradicción." if insat_kb else "No, es",)
print()

# 4) Refutación para probar a: KB {¬a}
print("4) Refutación por contradicción de a (BC {¬a})")
KB_mas_no_a = set(KB_CNF) | {frozenset({"¬a"})}
insat_contra_a = resolucion_insat_simple(KB_mas_no_a)

```

```

if insat_contra_a:
    print("Se obtuvo la cláusula vacía ( ) por lo tanto a pertenece a BC.")
else:
    print("No se obtuvo      no concluyente.")
print()

```

```

print("Cadena:")
print(" d y e (hechos)")
print(" R2: (~d ~e b) con d (~e b); con e b")
print(" R4: (~e c) con e c")
print(" R1: (~b ~c a) con b (~c a); con c a")
print(" Añadimos ¬a en BC genera contradicción ( )")

```

1) Encadenamiento hacia adelante
 Hechos iniciales: ['d', 'e']
 - b porque se verifica ['d', 'e']
 - c porque se verifica ['e']
 - a porque se verifica ['b', 'c']
 ¿a derivado?: sí

2) Encadenamiento hacia atrás (meta = 'a')
 - a se verifica con ['b', 'c']
 - b se verifica con ['d', 'e']
 - d es un hecho
 - e es un hecho
 - b se verifica
 - c se verifica con ['e']
 - c se verifica
 - a se verifica
 ¿a probado?: sí

3) ¿BC es inconsistente?
 Resultado: No, es CONSISTENTE porque no hay contradicción.

4) Refutación por contradicción de a (BC {¬a})
 No se obtuvo no concluyente.

Cadena:
 d y e (hechos)
 R2: (~d ~e b) con d (~e b); con e b
 R4: (~e c) con e c
 R1: (~b ~c a) con b (~c a); con c a
 Añadimos ¬a en BC genera contradicción ()

Bibliografía

Russell, S. & Norvig, P. (2004) *Inteligencia Artificial: Un Enfoque Moderno*. Pearson Educación S.A. (2a Ed.) Madrid, España

Poole, D. & Mackworth, A. (2023) *Artificial Intelligence: Foundations of Computational Agents*. Cambridge University Press (3a Ed.) Vancouver, Canada